

Crossy Road – sprawozdanie i workflow

Autorzy: Marcin Wolder, Stanisław Wojtas

Cel projektu

Celem było przygotowanie własnego środowiska typu Crossy Road, napisanie agenta rozwiązującego zadanie oraz pokazanie, że agent działa w sposób strategiczny, a nie losowy. Dodatkowo środowisko miało obsługiwać tryb graficzny, aby można było obserwować przebieg gry.

Wymagania sprawozdania

W raporcie powinny znaleźć się:

- opis środowiska Gymnasium,
- opis algorytmu uczącego,
- opis eksperymentów i wyników,
- informacja o trybie graficznym.

Instalacja

Najpierw warto zsynchronizować zależności w katalogu projektu.

```
In [12]: !uv sync
```

```
Resolved 82 packages in 3ms  
Audited 58 packages in 1ms
```

Opis środowiska

Środowisko `CrossyRoadEnv` udostępnia przestrzeń akcji `Discrete(5)`: ruch w górę, w dół, w lewo, w prawo oraz oczekiwanie. Obserwacja jest ciągła i ma postać wektora `Box(float32)`. Zawiera między innymi pozycję gracza, sygnały ryzyka, cechy najbliższego samochodu na każdym pasie oraz lokalną mapę zajętości wokół postaci.

Takie przedstawienie stanu sprawia, że agent może przewidywać ruch samochodów i podejmować decyzje na podstawie strategii, a nie tylko aktualnej pozycji na planszy.

Funkcja nagrody

Nagroda promuje ruch w stronę mety i karze zachowania nieefektywne. Agent dostaje dodatnią nagrodę za postęp, dużą nagrodę za ukończenie poziomu, karę za kolizję, niewielką karę krokową oraz dodatkowe korekty za ruch wstecz, ruchy boczne, stanie w miejscu i ignorowanie ryzyka.

Dzięki temu model ma uczyć się czekania na bezpieczne okno ruchu zamiast losowego klikania akcji.

Trening DQN

Do treningu użyto algorytmu DQN z biblioteki Stable-Baselines3. W aktualnej wersji projektu agent korzysta z pełnej przestrzeni pięciu akcji, co pozwala mu również manewrować w bok i cofać się, gdy wymaga tego sytuacja na drodze.

Parametry treningu:

- `timesteps = 100000`,
- `seed = 42`,
- `max_steps = 500`,
- `eval_freq = 10000`,
- `eval_episodes = 20`.

```
In [13]: Timesteps = 100_000  
SEED = 42  
MAX_STEPS = 500  
EVAL_FREQ = 10_000  
EVAL_EPISODES = 20
```

```
In [ ]: !uv run train.py --timesteps {Timesteps} --seed {SEED} --max-steps
```

Wyniki treningu

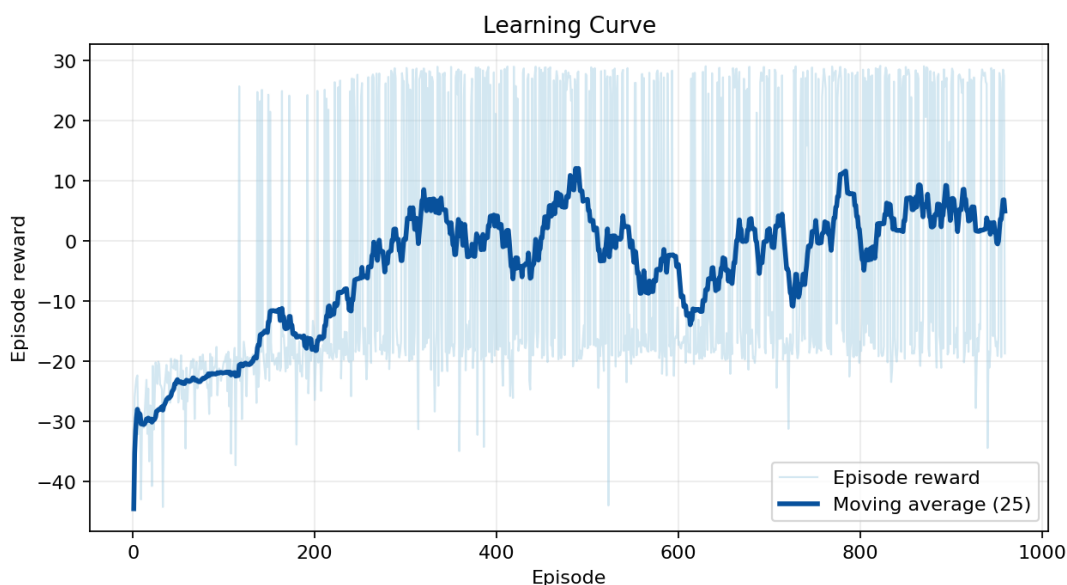
Ostatnie podsumowanie treningu zapisane w `artifacts/train_summary.json` wskazuje, że najlepszy checkpoint został wybrany automatycznie podczas ewaluacji okresowej.

Metryka	Wartość
Liczba epizodów treningowych	959
Średnia nagroda z ostatnich 20 epizodów	-0.5925
Najlepsza średnia nagroda z ewaluacji	8.439
Rozmiar przestrzeni akcji podczas treningu	5

Limit kroków

500

Krzywa uczenia została zapisana jako `artifacts/learning_curve.png`.



Ewaluacja

Końcowy model został porównany z bazą losową na 100 epizodach.

Najważniejsze metryki z `artifacts/eval_summary.json` są poniżej.

Metryka	Agent DQN	Polityka losowa
Win rate	0.51	0.00
Collision rate	0.47	0.67
Truncation rate	0.02	0.33
Średnia nagroda	5.13895	-
Średnia liczba kroków	98.35	341.33
Wait przy ryzykownym ruchu naprzód	0.2567	0.1956

Wniosek jest prosty: agent nie działa losowo i potrafi rozwiązać zadanie w ponad połowie epizodów. Jednocześnie nie jest jeszcze stabilny, bo nadal przegrywa część gier przez kolizje.

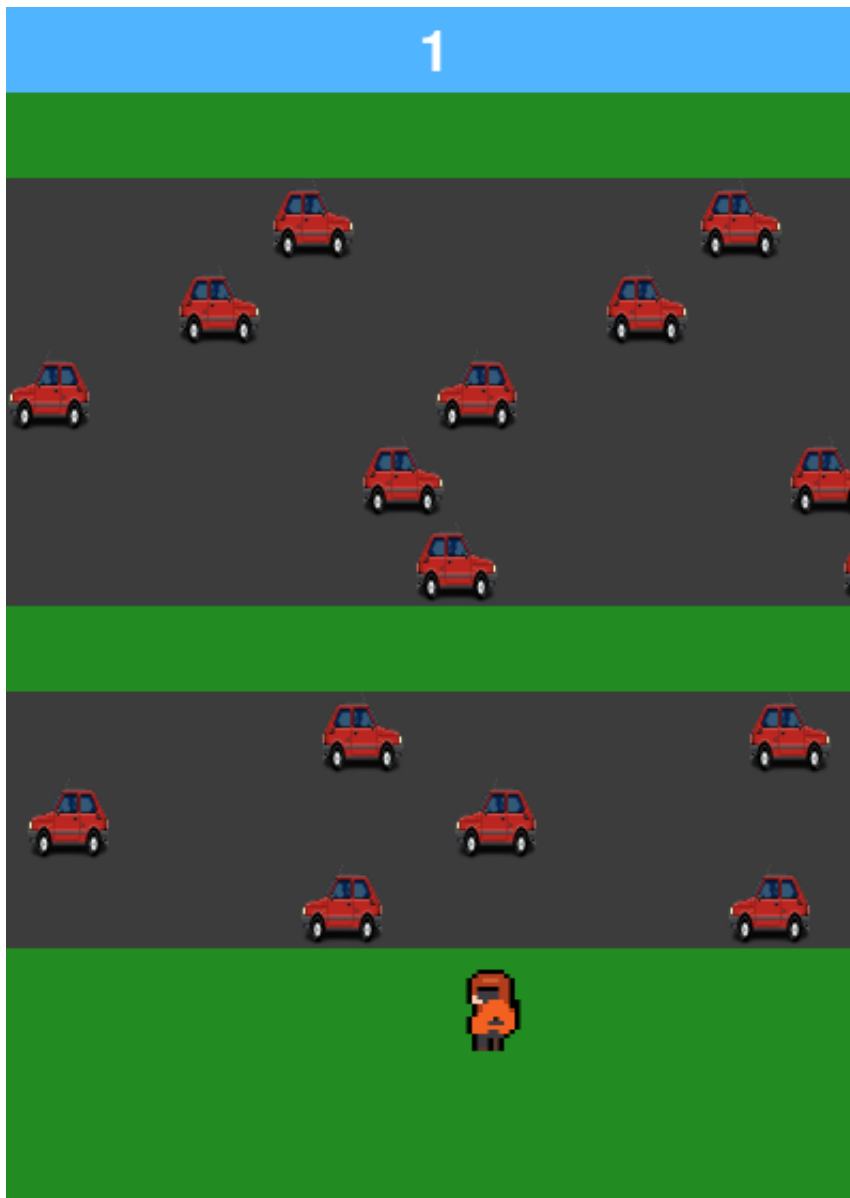
```
In [15]: !uv run evaluate.py --model artifacts/dqn_crossy_road.zip --episode
{
  "model": {
    "episodes": 100,
    "win_rate": 0.46,
    "collision_rate": 0.54,
    "truncation_rate": 0.0,
    "mean_reward": 2.8726000000000016,
```

```
"mean_steps": 84.03,  
"survival_time": 84.03,  
"action_counts": {  
  "up": 5285,  
  "down": 626,  
  "left": 1135,  
  "right": 1045,  
  "wait": 312  
},  
"action_rates": {  
  "up": 0.6289420445079138,  
  "down": 0.07449720337974533,  
  "left": 0.13507080804474592,  
  "right": 0.12436034749494228,  
  "wait": 0.03712959657265263  
},  
"wait_rate": 0.03712959657265263,  
"forward_risk_steps": 2606,  
"wait_risk_steps": 78,  
"wait_when_forward_risky_rate": 0.10514198004604758,  
"wait_when_wait_risky_rate": 0.0  
},  
"random_baseline": {  
  "episodes": 100,  
  "win_rate": 0.0,  
  "collision_rate": 0.73,  
  "truncation_rate": 0.27,  
  "mean_steps": 311.43,  
  "action_counts": {  
    "up": 6281,  
    "down": 5989,  
    "left": 6299,  
    "right": 6345,  
    "wait": 6229  
  },  
  "action_rates": {  
    "up": 0.20168256108916932,  
    "down": 0.19230645730982884,  
    "left": 0.20226054008926564,  
    "right": 0.20373759753395626,  
    "wait": 0.2000128439777799  
  },  
  "wait_rate": 0.2000128439777799,  
  "forward_risk_steps": 5843,  
  "wait_risk_steps": 44,  
  "wait_when_forward_risky_rate": 0.20212219750128357,  
  "wait_when_wait_risky_rate": 0.25  
},  
"delta_win_rate": 0.46,  
"delta_survival": -227.4  
}
```

Tryb graficzny

Środowisko obsługuje `render_mode="human"` oraz

`render_mode="rgb_array"` . W trybie `human` gra otwiera okno `pygame` , a w trybie `rgb_array` można pobierać klatki do zapisu lub analizy. Zamiast prostego kółka gracz jest rysowany sprite'em zależnym od kierunku ruchu.



Wnioski

Projekt spełnia wymagania zadania: powstało własne środowisko Gymnasium, agent DQN potrafi rozwiązywać zadanie w sposób nielosowy, a dodatkowo działa tryb graficzny. Najsilniejszym dowodem jakości modelu jest porównanie z bazą losową: model wygrywa często, podczas gdy polityka losowa nie wygrywa wcale.

Jeżeli chcesz wygenerować PDF z tego notebooka, użyj eksportu `nbconvert` z katalogu głównego repozytorium.